

NEDO積付アルゴリズムコンテスト:構築アルゴリズム改善の実装設計

TL;DR

(A) EP/EMSベースの増分候補生成が第一優先。全空間走査をやめ、荷物配置ごとに $O(n)$ で候補点を更新すれば、同じ3分/8秒予算内で試行回数が桁で増え、これが「あと数個」の非線形効果に直結する。到達可能性は「ロボット積付制約(robot-packable/top-down投入)」として候補点を事前フィルタする形で標準的に組み込める。実装は中規模、6週間で検証可能。

(B) precedence/blocking graphによる順序制御は第二優先。到達経路と下敷き禁止の両方を有向グラフにまとめ、トポロジカルソート+ヒューリスティックスコアで順序を決める。オフラインでは有効だが、オンライン(逐次到着)では近似運用になる。EP/EMSと併用すべき補助技術。
閾値型目的関数(足切り)は**観測不能閾値を当てにいかず**、頑健化(安全マージン・シナリオ平均・restricted candidate list)で「どの実現でも足切りを超える」配置数を狙うのが定石。配置数は単調に足切りを有利にするので、閾値の位置に関わらず「配置数最大化」が支配戦略になる。

Key Findings

候補点生成の刷新が最大のレバー。Crainic, Perboli & Tadei, "Extreme Point-Based Heuristics for Three-Dimensional Bin Packing" (*INFORMS Journal on Computing* 20(3):368–384, 2008, doi:10.1287/ijoc.1070.0250)のExtreme Point(EP)は、1配置あたり $O(n)$ で最大6個の新候補点を生成・挿入する増分アルゴリズムであり、全空間走査(グリッド全走査で $O(W \cdot D \cdot H)$)を置き換える。同論文が明言するとおりEPルールは "independent from the particular packing problem addressed and can handle additional constraints, such as fixing the position of the items" であり、本コンテストのような属性付き制約への拡張余地が構造的に用意されている。同じ時間で評価できる(位置×向き)組合せが激増する。

到達可能性は**robot-packable**制約として定式化されている。Martello-Pisinger-Vigoのrobot-packable変種、Wang & Hauser("Stable Bin Packing of Non-convex 3D Objects with a Robot Manipulator," IEEE ICRA 2019, pp.8698–8704, arXiv:1812.04093)のmanipulation feasibility、Ha et al.("An Online Packing Heuristic for the Three-Dimensional Container Loading Problem in Dynamic Environments and the Physical Internet," *Applications of Evolutionary Computation*, 2017)のOnlineBPHが「入口から目標位置まで運べるか(blocking problem)」を明示的に扱う。OnlineBPHは後続ベンチマークの記述によれば "follows the DBL order and evaluates all possible combinations of bins, items, EMS, and rotations across opened containers before selecting the optimal placement" というEMS駆動の逐次配置であり、EP/EMSの候補を「到達可能な候補」に制限する統合が標準。 [Ku](#)

属性制約(優先/ソフト/優先コンテナ)は候補点フィルタと順序制約で分離。幾何(EP/EMS)は純粋な空間管理に保ち、属性制約は「候補点admissibilityマスク」と「precedence辺」に外出しするのが、air-cargo ULD文献(Heßler, Hintsch & Wienkamp, "A Fast Optimization Approach For A Complex Real-Life 3D Multiple Bin Size Bin Packing Problem," arXiv:2410.01445, 2024; Paquay et al. 2016)の標準的な設計。Heßler et al.は $\text{stackable} \in \{0,1\}$ 等の属性をextreme-point insertion heuristicの候補判定に直接組み込んでいる。

precedence graphは多降ろし(multi-drop)制約と同型。「後から降ろす荷物が手前を塞がない」制約は、本コンテストの「後ろにしか置けない」到達制約と数学的に同じLIFO/precedence構造。Junqueira, Morabito & Yamashita, "MIP-based approaches for the container loading problem with multi-drop constraints" (*Annals of Operations Research* 199(1):51–75, 2012, doi:10.1007/s10479-011-0942-z)、および同著者らの積載安定性・耐

荷重を扱うMIP(*Computers & Operations Research* 39(1):74-85, 2012)にMIP/ヒューリスティック双方の定式化がある。

閾値未知の頑健化は、chance-constraint→robust近似(Ben-Tal/Nemirovski系)やGRASPのrestricted candidate listのように「保守側マージン」「複数実現での最悪/平均評価」で扱う一般論が確立している。

Details

(A) Extreme Point (EP) + Empty Maximal Space (EMS) の増分更新

概要

候補配置点の集合を、全空間走査ではなく「配置済み荷物が作る空きコーナー」だけに絞って**増分的に**維持する。EPは点ベース(Crainic et al. 2008)、EMSは直方体(極大空きボックス)ベース(Lai & Chan 1997; Parreño, Alvarez-Valdés, Tamarit & Oliveira, "A Maximal-Space Algorithm for the Container Loading Problem," *INFORMS J. Computing* 20(3):412-422, 2008; Gonçalves & Resende, "A biased random key genetic algorithm for 2D and 3D bin packing problems," *International Journal of Production Economics* 145(2):500-510, 2013, doi:10.1016/j.ijpe.2013.04.019)。両者は相補的で、EPは「置ける点」、EMSは「その点で使える余白(residual space)」を与える。本コンテストのように候補評価が高コスト(物理判定・経路判定)な場合、候補集合を小さく保つEP/EMSは試行回数を最大化する。

EPの標準定義と増分更新(擬似コード)

荷物kをサイズ(w,d,h)で左後下(left-back-down)コーナー(x,y,z)に置くと、3つの種コーナー点

A=(x+w, y, z) (X方向端)

B=(x, y+d, z) (Y方向端)

C=(x, y, z+h) (Z方向端)

を、それぞれが動いていない2軸の**負方向**に投影して、最大6個の新EPを得る(投影表):

種コーナー 投影方向1 投影方向2

A=(x+w,y,z) -Y -Z

B=(x,y+d,z) -X -Z

C=(x,y,z+h) -X -Y

投影は「kとコンテナ壁の間にある既積み荷物すべて」に対して行い、**最も近い**衝突面で止める。空コンテナなら(0,0,0)に置き、(w,0,0),(0,d,0),(0,0,h)の3点を生む。EPリストは(z,y,x)降順で保持。各挿入は全荷物を1回走査($O(n)$)+6点の二分挿入で、**1配置あたり $O(n)$** (Crainic et al. 2008, Theorem 1)。

各EPには**Residual Space(RS)** = (RSx, RSy, RSz)を持たせる。RSは「そのEPから、各軸方向にコンテナ壁または最近接の既積み荷物までの距離」。生成時は各軸のコンテナ壁までの距離で初期化し、荷物追加のたびに全EPのRSを1パス($O(n)$)で「min(現RS, 近接面までの距離)」に縮める(論文Algorithm 2)。荷物kがEP eに入るかは $w \leq RSx_e$ and $d \leq RSy_e$ and $h \leq RSz_e$ で判定でき、適合度 $fb = (RSx-w)+(RSy-d)+(RSz-h)$ で最小の余りを選ぶ(best-fit)。



```

place_item(k, (x,y,z), EPL, placed):
    seeds = [ (x+w, y, z, ['-Y','-Z']),
              (x, y+d, z, ['-X','-Z']),
              (x, y, z+h, ['-X','-Y']) ]
    new_eps = []
    for (px,py,pz,dirs) in seeds:
        for dir in dirs:
            q = project(px,py,pz, dir, placed, container) # 最近接面まで滑らす
            if q is valid: new_eps.append(q)
    for e in EPL:          # RS更新 O(n)
        update_residual_space(e, k_box)
    for q in new_eps:
        q.RS = init_residual_space(q, placed, container)
        binary_insert(EPL, q, key=(-z,-y,-x))
    remove_covered_eps(EPL)      # 他EPに内包される点を除去

```

EMSは点ではなく極大空きボックスを保持し、Lai-Chanの**difference process**で更新する。EMS $S = (x_1, y_1, z_1) - (x_2, y_2, z_2)$ と交差する箱 $B = (bx_1..bx_2, by_1..by_2, bz_1..bz_2)$ から最大6個の新EMSを生成:

- ⊗
1. $(x_1, y_1, z_1) - (bx_1, y_2, z_2)$ # $bx_1 > x_1$ のとき
 2. $(bx_2, y_1, z_1) - (x_2, y_2, z_2)$ # $bx_2 < x_2$
 3. $(x_1, y_1, z_1) - (x_2, by_1, z_2)$ # $by_1 > y_1$
 4. $(x_1, by_2, z_1) - (x_2, y_2, z_2)$ # $by_2 < y_2$
 5. $(x_1, y_1, z_1) - (x_2, y_2, bz_1)$ # $bz_1 > z_1$
 6. $(x_1, y_1, bz_1) - (x_2, y_2, z_2)$ # $bz_2 < z_2$

各新EMSは他2軸ではSの全域を保つ(だから極大)。更新後、

内包除去: $S_i \subseteq S_j$ (全6面が S_j に含まれる) なら S_i を削除。

サイズ除去: 残り荷物の最小寸法($w_{min}, d_{min}, h_{min}$)より小さいEMSを削除(Gonçalves-Resende 2013の高速化)。

EMSの選択はコンテナ隅への距離を非減少ベクトルで辞書順比較(Parreño et al. 2008)、またはGonçalves & Resendeの**DFTRC(Distance to the Front-Top-Right Corner)**規則 — 「箱をコンテナの最大座標隅から最も遠くなる空きスペースに置く (the heuristic will always place the box in the empty space such that it maximizes the distance of the box to the maximal coordinates of the container)」 — で行う。 [UV](#)

到達可能性制約(搬入経路)の統合 — 標準的対処

素朴なEP/EMSは「その位置に置けるか」のみを見て「そこまで運べるか」を見ないため、そのままだと不整合が起きる。標準的な3段階の対処:

候補点を到達可能性でフィルタ(admissibility mask)。 EP/EMSが出す候補位置を、そのまま採用せず「入口から目標へのL字経路が既積み荷物・壁・棚と1.5cm以内で干渉しないか」を判定してから採用集合に入れる。Ha et al.(2017)のOnlineBPHはまさにこの「blocking problem(入口から目標へ運ぶ途中で既積み荷物に塞がれ

る)」を明示的に候補フィルタとして扱う。Wang & Hauser(2019)の`isManipFeasible`も「上空の退避位置から目標へ補間した経路(top-down placement trajectory)が衝突しないか」を候補ごとに判定する同型の手続き。**robot-packable順序の強制**。Martello-Pisinger-Vigoのrobot-packable制約=「各荷物は既積み荷物すべての前・右・上のいずれかに置かれる」を守れば、投入経路が構成的に確保される。本コンテストのL字経路(Y→X、上方8cmから)はこの一般化であり、候補生成時に「後方/側方が既に塞がれている位置」を除外すればよい。[KU](#)

フォールバック探索。多くの候補が到達不能で棄却されるため、Wang & Hauserは「まず低次元(3D)で高確率に解がある空間を探し、なければ高次元探索や配置順の入替にフォールバック」する(彼らの実測では、優先探索は常時5D探索より約15倍高速)。本コンテストのonline 8秒でも、EP候補をrobot-packableでソート→到達判定→最初の合格点(first-fit)を採る戦略で、判定回数を抑えつつ確実に間に合わせる。

重要な注意: 到達判定(pybulletでのL字経路シミュレーション)は物理判定より重い可能性があるため、(a)まず軽量の幾何判定(掃引直方体=swept volumeと既積みAABBの交差)で大半を弾き、(b)通過したものだけpybulletで精密判定、の二段構えにする。掃引直方体はL字経路(上空8cm→Y移動→X移動→降下)を数個のAABBで近似すればnumpyでベクトル化できる。

属性制約(優先コンテナ・下敷き禁止)の統合

EP/EMSは幾何のみを扱うので、属性は**分離して**足す:

優先コンテナ制約: コンテナごとに「受入可能な荷物属性集合」を持たせ、候補生成前に荷物→コンテナのマッピングでフィルタ(幾何に触れない)。

下敷き禁止(優先/ソフトの上に載せない): これは「支持関係」の制約。候補点admissibilityで「この候補位置の直下の支持荷物にnon-stackable属性(優先/ソフト)が含まれるなら棄却」とする。Heßler et al.(2024)のair-cargo ULDアルゴリズムがまさに `stackable` 属性を候補フィルタに組み込む形で実装しており、EPフレームワークと両立する。

実装上は、各EP候補に対して `admissible(ep, item) = geometry_ok(ep, item) and container_ok(item) and support_ok(ep, item) and reachable(ep, item)` という合成述語を通すだけ。幾何コアは無改造。

実装規模の目安: 中

既存のnaive全空間走査を、EPリスト管理クラス(挿入/RS更新/内包除去)に差し替える。コア(EP生成+RS)は数百行。既存の物理判定・経路判定・属性判定は`admissible`述語として再利用でき、大改修は不要。GitHubに参考実装が複数ある(rmercado6/Extreme-Point-Heuristics, dasvision0212/3D-Bin-Packing-Problem-with-BRKGA のEMS difference process, enzorui/3dbinpacking のpivot方式)。6週間で実装+検証可能。

期待できる効果の性質

主効果=計算が速くなる → 同予算で試行回数増 → 配置数が増える。候補集合が全空間走査の数千点からEP/EMSの数十点に減り、1候補あたりの重い判定(物理・経路)の回数が激減する。浮いた予算をランダム化リスタート(Heßler et al.のRandomized Greedy Search、Parreñoのreactive GRASP)に回すと、同じ3分で多数の構築試行ができ、最良解を採れる。

充填率も副次的に上がるが、本コンテストでは**足切り突破(配置数)**への寄与が本命。

(B) Precedence graph / blocking graph ベースの配置順序制御

概要

「荷物*i*は*j*より先に置かないと物理的/経路的に不可能」という依存関係を有向辺 $i \rightarrow j$ で表し、トポロジカルソートで実行可能な配置順序を作る。到達制約(奥・下から先)と下敷き禁止(支持側を先)の両方が自然に辺として表現できる。多降ろし(multi-drop)コンテナ積付の標準技術(Junqueira et al. 2012; de Queiroz & Miyazawa 2013)と同型。

precedence graphの構築(標準的方法)

2種類の辺を張る:

到達/支持辺(geometry-driven): 目標レイアウトが与えられたとき(オフライン)、「*i*を置いた後だと*j*の搬入経路が*i*に塞がれる」なら $j \rightarrow i$ (*j*を先に)。同様に「*j*が*i*の真下で支持する」なら $j \rightarrow i$ 。L字経路($Y \rightarrow X$)なら、*i*より入口側(Y 大 or X 大側)かつ経路上にある荷物が先。

属性辺(attribute-driven): 優先手荷物は指定コンテナへ、ソフト/優先の上には載せない → 「支持する荷物(下)を先、載る荷物を後」。下敷き禁止は「優先/ソフトは必ず最上段側」 = それらに向かう辺を最後に。



```
build_precedence(layout):
```

```
    G = empty_digraph(items)
```

```
    for i, j in pairs(items):
```

```
        if blocks_entry(j, i, Lshape_path): G.add_edge(j, i) # jが先
```

```
        if supports(j, i): G.add_edge(j, i) # 下が先
```

```
    for s in soft_or_priority_items:
```

```
        for k on_top_candidates(s): forbid_edge s->under(k) # 下敷き禁止
```

```
    return G
```

順序決定(トポロジカルソート + ヒューリスティックスコア)

純粋なトポロジカルソートは複数の妥当順序を許すので、Kahnのアルゴリズムを**優先度付きキュー**で回し、「今置ける荷物(入次数0)」の中からヒューリスティックスコア最大を選ぶ:



```
topo_greedy(G, score):
```

```
    ready = { v : indeg(v)==0 }    # 優先度キュー
```

```
    order = []
```

```
    while ready:
```

```
        v = argmax_{u in ready} score(u) # 例: 体積大・底面積大・重い・優先属性
```

```
        order.append(v); remove v
```

```
        for w in succ(v):
```

```
            indeg[w]-=1
```

```
            if indeg[w]==0: ready.add(w)
```

```
    return order
```

scoreは「大きい/重い/低く置くべき荷物を先」(重心を下げ、下敷き禁止と整合)が定石。これは構築ヒューリスティック(EP/EMS)に渡す**荷物順序**を決め、EP/EMSが各荷物の位置を決める、という二段構成になる(Gonçalves & Resendeのsequence→placement decoderと同じ分業)。

オンライン(逐次到着)環境での近似適用

荷物が逐次到着し全体が見えない場合、完全なprecedence graphは作れない。近似運用:

look-aheadウィンドウ: 見えている範囲(EBS/メイクエリアのバッファ)だけで部分precedence graphを作り、その中で順序最適化。Zhao et al.のPCT-reorder(先に来る荷物が後の荷物で開いた空間を使わない、という順序依存制約を守る仮想並べ替え)が該当する — "an earlier-arriving parcel cannot occupy a space made available only by the placement of a later-arriving one" という制約を守りつつ仮想的に並べ替える。 [arxiv](#)

保守的配置: 未来が不明なら「後から到達不能にしない」位置を選ぶ。具体的には、各候補位置について「この位置を今埋めると、まだ来ていない荷物の到達可能空間(入口側・上側)をどれだけ塞ぐか」をペナルティ化し、塞ぎの少ない位置を優先。これはBonet Filella, Trivella & Corman(2022, *European Journal of Operational Research*)のsoft unloading constraint(ハード制約でなくペナルティ化)と同じ発想。

online 8秒では完全グラフ再構築は避け、到着荷物1個について「現在の到達可能EP候補の中でrobot-packableかつ下敷き禁止に反しない位置をfirst-fit」で十分。precedence graphはオフライン3分側で本領を発揮する。

実装規模の目安: 小〜中

グラフ構築+Kahn+優先度キューは標準的で、networkx不使用でも数百行。難所は [blocks_entry](#) (L字経路の相互ブロック判定)だが、これは(A)の到達判定ロジックを再利用できる。単独では効果が限定的で、(A)の上に載せる補助技術。6週間内で(A)の後に追加実装可能。

期待できる効果の性質

配置順序が変わることで、到達不能で棄却される荷物が減る → 配置数が増える。 特に「奥/下の荷物を確実に先に置く」ことで、後から来る荷物が入口に塞がれて置けなくなる事故を防ぐ。重心も下がりやすい(重い/大きいを先=下に置く順序と整合)。

副次: 閾値型目的関数(観測不能な足切り)への一般的戦略

足切り閾値を推測・逆算するのは規約違反かつ過学習リスクが高い。一般論としての定石:

保守的安全マージン: 「閾値を超える」ではなく「閾値+マージンを超える」配置数を目標に設計。最適化理論ではchance constraint $P(\text{達成}) \geq 1 - \epsilon$ をrobust近似(Ben-Tal & Nemirovski)で「不確実性集合内の最悪ケースでも達成」に置換するのが標準。閾値が未知なら不確実性集合を閾値の取りうる範囲として扱う。

単調性を利用: 足切りは「配置数が多いほど超えやすい」単調構造なので、閾値を当てにいく必要がなく、とにかく配置数を最大化すれば頑健に足切り側を制する。閾値の位置に関わらず配置数最大化が支配戦略。

複数シナリオでの評価: 提出前にローカルで複数の荷物順・乱数シードで構築し、「最悪シナリオでの配置数」を見る(robust optimizationのmin-max、またはGRASPのように多数試行の最良を採る)。

restricted candidate list(RCL): GRASPのように「上位数%の候補からランダム選択」して多様な構築解を作り、足切りを安定して超える解を採る。

Recommendations

第1段階(週1〜3): EP/EMS増分更新の導入。 全空間走査をEPリスト管理に置換。 [admissible](#) 述語(幾何・コンテナ・支持・到達)で既存判定を再利用。到達判定は掃引直方体の軽量プレフィルタ→pybullet精密判定の二段。ベンチマーク: 同一時間予算での構築試行回数と平均配置数。 **判断基準:** 試行回数が5倍以上、配置数中央値が現状+1個以上なら成功、public scoreで検証(過去に配置数+0.88ppでscore+2.41の実績があるため、+1個でも大きく効く見込み)。

第2段階(週3〜4): ランダム化リスタート。 EPコアが速くなった余剰予算をRandomized Greedy Search/reactive GRASPに投入(offline 3分で多数試行、best解採用)。**判断基準:** 配置数の分布上側(最良試行)が押し上がるか。

第3段階(週4〜5): precedence graph順序制御。 オフライン側でtopo_greedy順序をEP/EMSに供給。到達不能棄却率を計測。**判断基準:** 棄却率が下がり配置数が増えるか。効果が薄ければオンライン側は現状のfirst-fitに留める。

第4段階(週5〜6): 頑健化と検証。 複数シード・複数荷物順でのローカル評価、配置数最大化を一貫目標に。閾値推測は行わない。

やらないこと: 隠しパラメータの逆算。理論的に優れていても3分/8秒に収まらない手法(厳密MIP、重い分枝限定、フルDRL学習)は本番採用しない — ただしオフライン学習済みDRL(PCT等、alexfrom0815/Online-3D-BPP-PCT に実装あり)は推論が軽いので将来検討余地あり。

Caveats

到達判定のコストが律速になりうる。 EP/EMSで候補を減らしても、L字経路のpybullet判定が重ければonline 8秒を圧迫する。掃引直方体プレフィルタの精度と速度が成否を分ける。実測で判定回数の予算配分を必ず確認すること。

EPの投影は「上からの重力落下」を前提としない純幾何。 本コンテストは物理演算で定着確認するため、EPが出す点が物理的に安定とは限らない。RSベースのbest-fitに加え、支持面積チェック(既存の有効パラメータ=支持接触面の重心オフセット許容量)を候補フィルタに残すこと。

precedence graphのオンライン適用は原理的に近似。 全荷物が見えない以上、最適順序は保証されない。look-aheadウィンドウのサイズと、コンテスト仕様上どれだけ先が見えるかに依存する(仕様要確認)。

EMSの内包除去・サイズ除去を怠るとEMSリストが膨張し、増分更新の速度優位が消える。 除去処理は必須。Gonçalves & Resende(2013)とLai & Chan(1997)の原論文の厳密な数式は有料で、上記EMS 6分割の座標式はParreño et al.のオープンな技術報告(tr03-07)とオープン実装から挙動を確認しつつ幾何的定義から再構成したもの。実装時はオープンソース実装(dasvision0212のBRKGA等)で挙動を照合することを推奨。同様に、Crainic et al.のAlgorithm 1/2の行単位擬似コードは論文Appendixにあり、本設計の擬似コードは本文の投影規則・RS定義・ $O(n)$ 証明から構成した等価版である。

本コンテスト固有のAKE/AKN形状(非直方体・斜め壁)には、Heßler et al.(2024)の「moving extreme points(斜め面用にEPを移動させ、上部の斜面下の充填密度を上げる)」が直接応用でき、EPフレームワークはこの拡張に対応可能。